Structure of Computer Systems

Virtual Memory Simulator

Contents

1. Introduction

1.1 Context

In modern computing systems, virtual memory plays a critical role in managing memory efficiently and securely. By separating physical and logical memory, it allows programs to run as if they have access to a large, continuous block of memory, regardless of the actual physical memory size. This mechanism enables multi-tasking, memory isolation, and efficient resource utilization.

However, the internal operations of virtual memory systems—such as page lookup, page fault handling, and page replacement—are not directly visible to students, which can make the concept challenging to fully grasp. A simulator that visualizes these processes can bridge the gap between theoretical knowledge and practical understanding.

1.2 Problem

Students learning computer architecture and operating systems often struggle with:

- Understanding how the system locates pages in memory.
- What happens when a requested page is not present in main memory.
- How and why certain pages are replaced when memory is full.
- Comparing the impact of different page replacement algorithms on system performance.

Although the theory is well covered in textbooks, the lack of interactive, educational visualization tools makes the learning process less intuitive.

1.3 Motivation

The motivation for this project stems from the need for a practical and visual way to teach virtual memory concepts. By creating an interactive simulator, students can:

- Observe memory operations step by step.
- Experiment with different page replacement algorithms.
- Understand how page faults affect performance.
- Strengthen their conceptual understanding through visual feedback.

1.4 Objectives

The main objectives of the project are:

- To develop a graphical virtual memory simulator that demonstrates how paging and page replacement work.
- To implement mechanisms for:
  - Page lookup in memory.
  - Handling page faults.
  - Replacing pages when memory is full.
- To support multiple page replacement algorithms such as:
  - FIFO (First In First Out)
  - LRU (Least Recently Used)
  - Optimal replacement
- To provide visual statistics (e.g., hit/miss ratio, number of faults) that help analyze system performance.
- To create an intuitive user interface suitable for educational use.

1.5 Project Timeline / Plan

Week	Target	Description
4	**Specification & research**	Define the problem and make research on similar projects.
6	**Core structure & Simulation Engine**	Implement page table, memory representation, page lookup and page fault handling
8	**Replacement Algorithms**	Develop FIFO, LRU, and Optimal algorithms.
10	**GUI Development**	Design and implement the graphical interface and connect the simulation with the backend.
12	**Final Testing & Documentation**	Finalize project (add possible optimizations and enhancements), polish UI, and prepare report/presentation.

2. Bibliographic Study

2.1 Critical Analysis of Similar Existing Solutions

Over the years, several educational tools and academic projects have been developed to illustrate how virtual memory works. These can generally be divided into two main categories: command-line simulators and graphical simulators.

In [1], the authors propose a command-line based virtual memory simulator focused on demonstrating how FIFO page replacement works. This approach is simple and effective for understanding the basic flow of page loading and replacement. However, because it relies solely on text output, it is not interactive and provides no real-time visualization, making it less engaging and harder for beginners to follow.

In [2], a basic GUI simulator was introduced to give students a visual understanding of page replacement. The graphical interface allows users to observe the state of physical memory as new page requests arrive. While this solution improves comprehension compared to command-line tools, it typically supports only a single algorithm and offers limited customization.

In [3], a more advanced GUI tool was presented, implementing multiple page replacement algorithms, including FIFO, LRU, and Optimal. It provides visual feedback and performance metrics such as hit/miss ratio and number of page faults. Although this approach is more educationally effective, it still has some drawbacks such as limited flexibility, lack of support for different operating environments, and restricted control over simulation parameters.

In summary, existing solutions address parts of the problem but rarely offer a fully flexible, interactive, and educational tool. This project aims to fill that gap by combining multiple algorithms, interactivity, and clear visualization.

2.2 Theoretical Concepts

To implement a virtual memory simulator that reflects real-world behavior, several key concepts must be well understood and applied:

- **Virtual memory**: Allows processes to use more memory than physically available by mapping virtual addresses to physical memory.
- **Page table**: A data structure used to translate virtual addresses into physical addresses.
- **Page fault**: Occurs when a requested page is not in main memory, triggering a load from secondary storage.
- **Page replacement algorithm**: Determines which page to remove from memory when space is needed (e.g., FIFO, LRU, Optimal).

- **Memory management unit (MMU)**: The hardware component that supports address translation.

These theoretical concepts are the foundation for the simulator. Understanding them ensures that the implementation closely mirrors how virtual memory is managed in actual operating systems.

2.3 How Others Solve the Problem

Existing solutions typically follow one of two approaches:

Algorithmic Simulation: Often text-based programs that process a sequence of page references and print step-by-step results. Good for demonstrating the logic of page replacement but lack interactivity and visualization.

Graphical Visualization: Simulators that provide a GUI displaying frames, requested pages, and replacement events in real time. They enhance comprehension by allowing students to observe algorithm behavior dynamically, but many are limited to one algorithm or lack flexibility for customization.

In [2], the GUI simulator offers a straightforward visualization of how pages are loaded and replaced. In [3], the authors improve upon this by supporting multiple algorithms, showing how different strategies affect performance. These works confirm that visualization is a powerful tool for learning, but also highlight the need for a more adaptable, educationally focused simulator.

3. Analysis

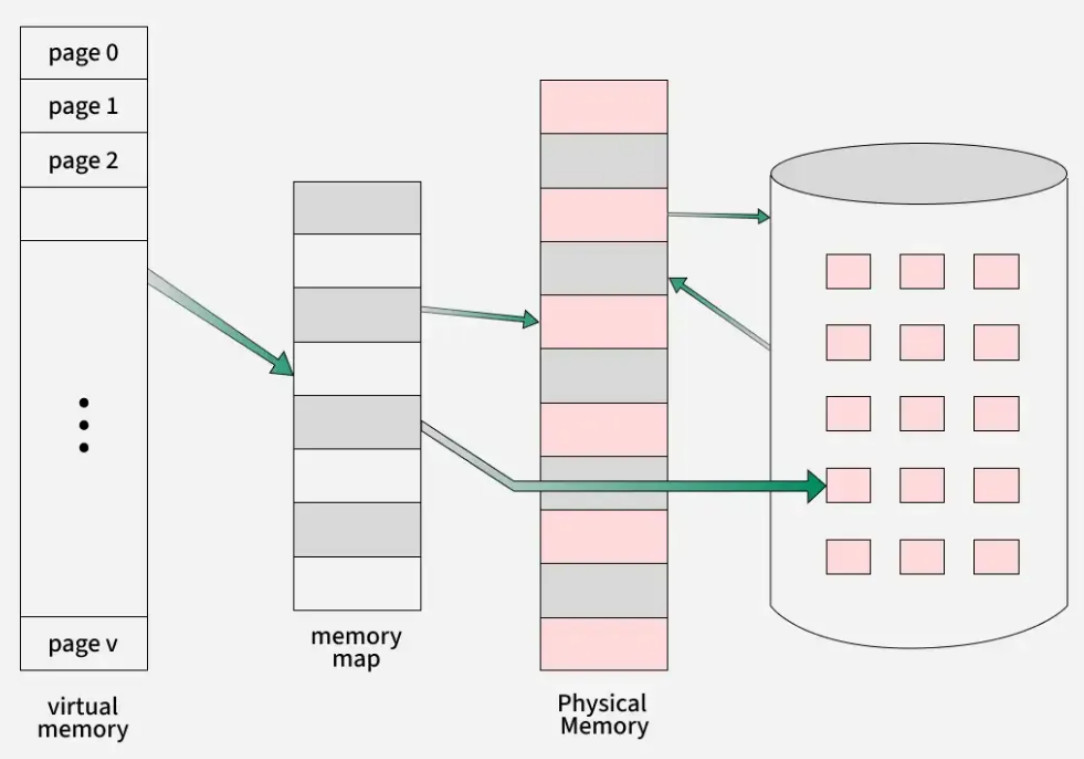

Figure 1. Virtual memory overview diagram.

3.1 Algorithms

The simulator supports three classical page replacement algorithms used in operating systems. These algorithms control which page is removed from physical memory when a page fault occurs and no free frame is available.

3.1.1 FIFO (First-In, First-Out)

FIFO removes the page that has been in memory the longest. A simple queue structure tracks the insertion order of pages.

Advantages:

- Easy to implement.
- Low overhead.

Disadvantages:

- Does not consider usage frequency.
- Vulnerable to Belady's anomaly.

3.1.2 LRU (Least Recently Used)

LRU removes the page that has not been used for the longest period of time. It attempts to approximate real program behavior by assuming that recently used pages are more likely to be used again.

Advantages:

- Generally good performance.
- Matches memory access patterns in real workloads.

Disadvantages:

- Requires tracking recent access times.
- More overhead than FIFO.

3.1.3 Optimal (OPT / MIN)

Optimal replacement removes the page whose next use is farthest in the future. It is impossible to implement in a real system but provides a baseline for algorithm comparison.

Advantages:

- Produces the minimum number of page faults.

Disadvantages:

- Requires knowledge of future accesses.
- Only usable in simulators.

Figure 1 (shown at the beginning of this chapter) illustrates the main components involved in virtual memory systems: the CPU, TLB, page table, physical memory, and secondary storage. These concepts form the foundation of the simulator's design.

3.2 Hardware and Software Mechanisms

Virtual memory involves hardware support within the CPU and Memory Management Unit (MMU), combined with operating system mechanisms for handling page faults and managing the page table.

3.2.1 Key Hardware Components

- **CPU** – Issues virtual memory requests.
- **MMU** – Translates virtual addresses to physical ones.
- **TLB** – A cache for recently used page translations.
- **Main Memory** – Holds resident pages.
- **Secondary Storage** – Stores non-resident pages.

3.2.2 Address Translation

A virtual address is divided into:

- a **page number**,
- an **offset**.

The page table maps the virtual page number to a physical frame number. If the page is not present in memory, a page fault occurs.

3.2.3 Page Table Entry (PTE)

Each PTE includes:

- frame number,
- present bit,
- reference bit,
- dirty bit,
- protection flags.

3.2.4 Page Fault Handling

The steps are:

1. The CPU traps to the OS.
2. The OS locates the page on disk.
3. If a free frame exists, it loads the page.
4. Otherwise, a replacement algorithm selects a victim.
5. Page table entries are updated.
6. Execution continues.

3.2.5 TLB Interaction

1. Check the TLB.
2. On TLB miss, check the page table.
3. On page table miss, trigger page fault.

3.2.6 Address Translation / Page Fault Diagram

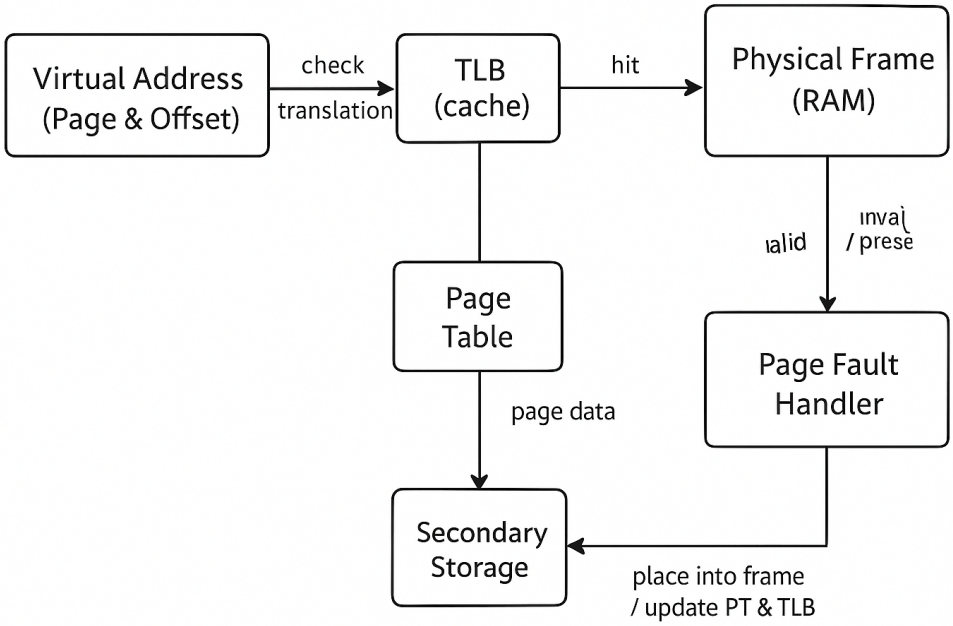

Figure 2. Address translation and page fault handling process.

Figure 2 illustrates the decision path taken by the processor and OS when translating virtual addresses and resolving page faults.

4. Design

4.1 System Architecture

The simulator uses a **Model-View-Controller (MVC)** structure:

- **Model (Simulation Engine)** – implements page table, frames, and replacement algorithms (FIFO, LRU, Optimal) and gathers statistics.
- **View (User Interface)** – displays frames, page faults, and statistics and provides controls for input.
- **Controller (SimulationController)** – connects UI actions to the Model and triggers view updates.

The core loop is:

1. Controller takes the next page from the reference string.
2. Model checks if it is in memory and, if needed, performs replacement using the selected policy.
3. Model updates statistics and state.
4. Controller instructs the View to redraw memory and metrics.

Class Diagram

The Class Diagram shows:

- **SimulationController** linking the UI, SimulationEngine, and StatisticsTracker.
- **SimulationEngine** using a **PageTable**, **Frame** objects, and a **ReplacementPolicy** hierarchy (FIFO, LRU, Optimal).
- Clear separation between control, data, and visualization (MVC).

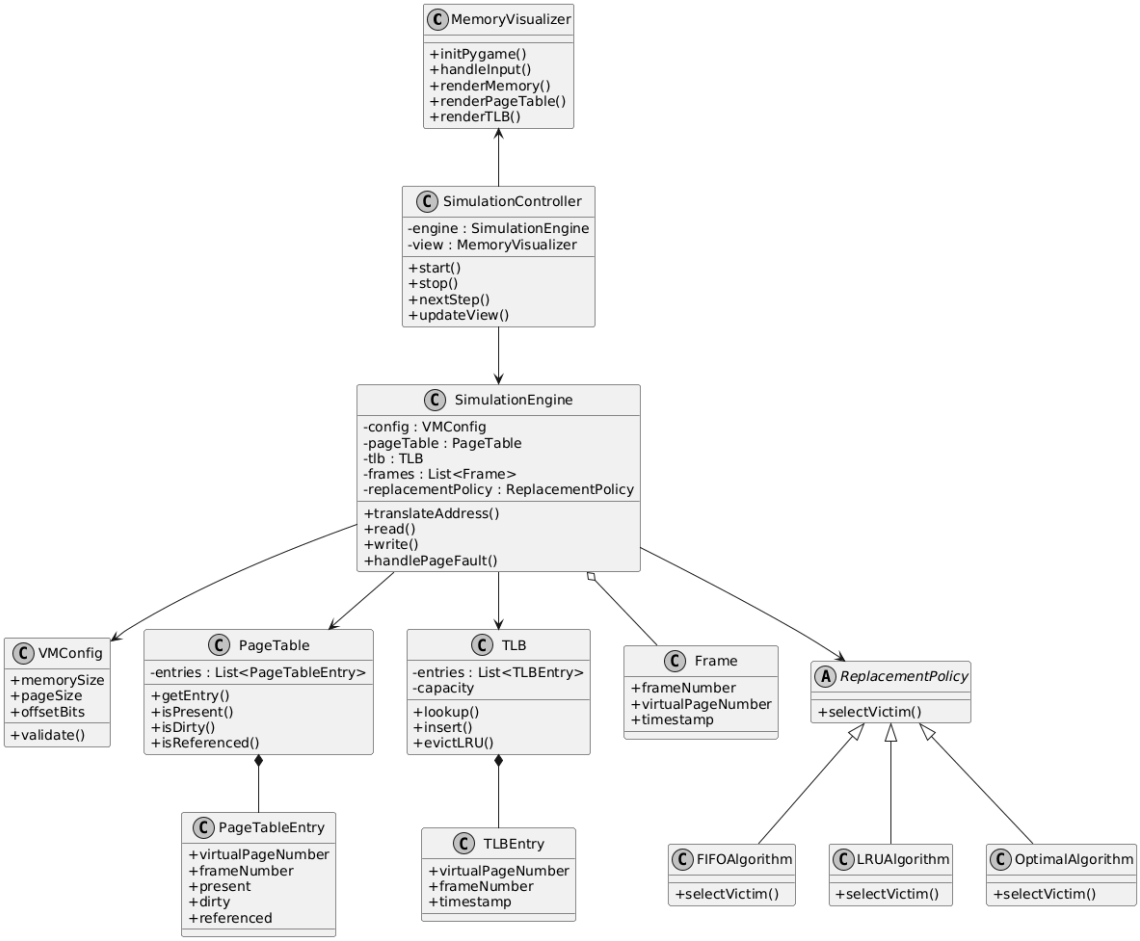

Figure 1: Class Diagram of the Virtual Memory Simulator

Use Case Diagram

The Use Case Diagram models the main user actions:

- Select algorithm,
- Input reference string,
- Start / pause / reset simulation,
- View results and statistics.

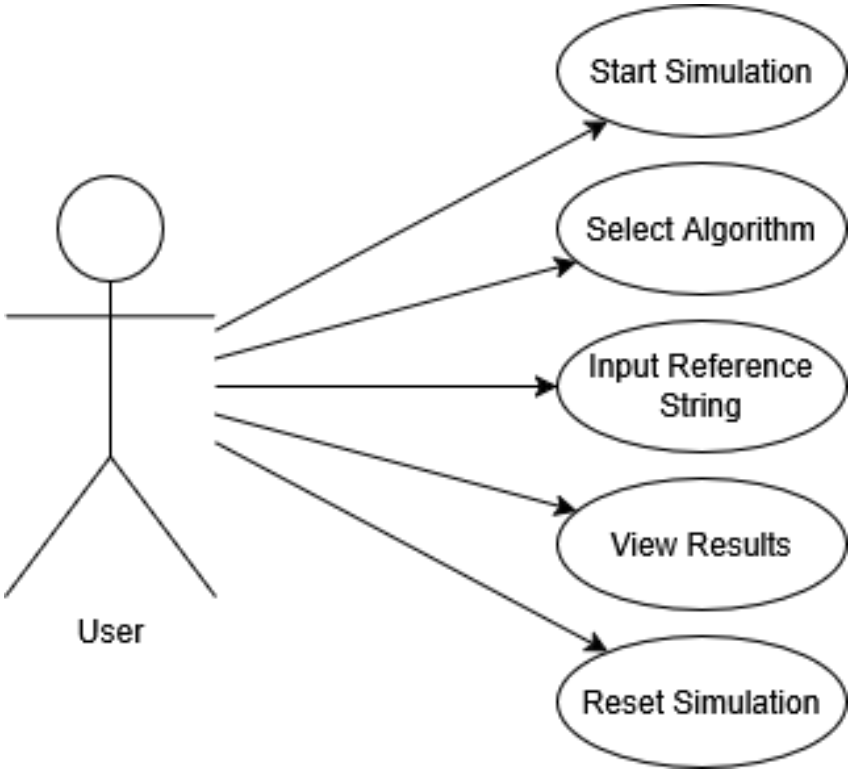

Figure 2: Use Case Diagram of the Virtual Memory Simulator

Sequence Diagram

The Sequence Diagram illustrates one page request:

- User sends a page request through the UI.
- Controller forwards it to the SimulationEngine.
- Engine may call the ReplacementPolicy to select a victim.
- After updating state and statistics, the Controller triggers a UI refresh.

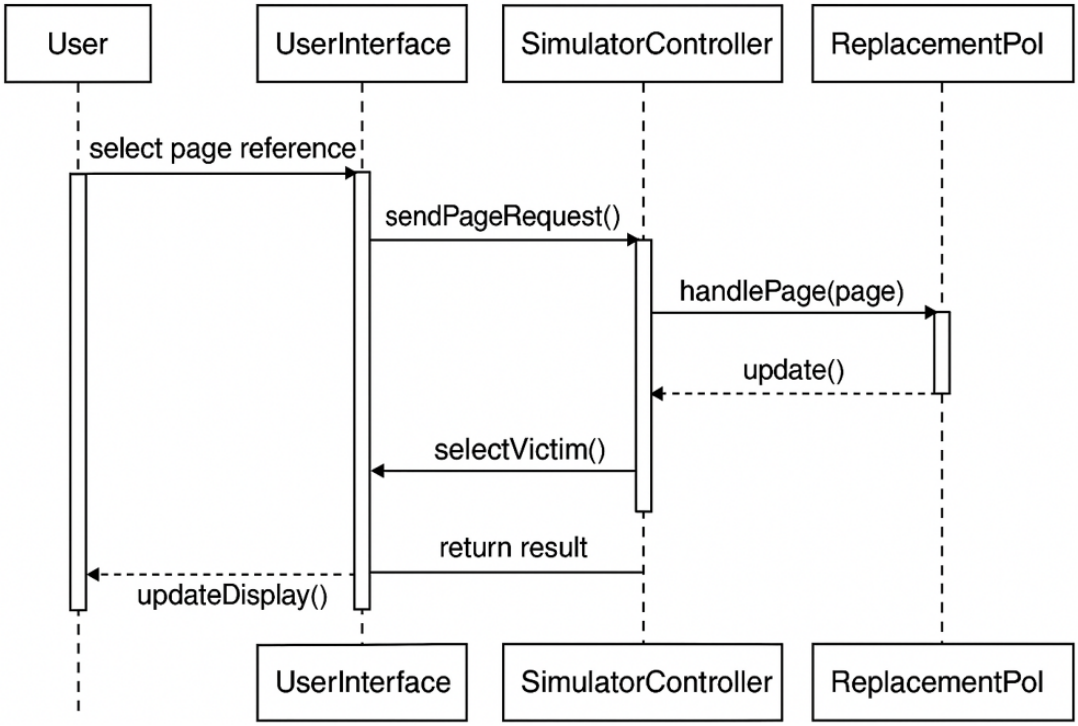

Figure 3: Sequence Diagram Showing Page Request Handling

4.2 Component Overview

Below is a short description of each main component, in line with the MVC structure.

4.2.1 SimulationController

Role: Central coordinator (Controller).

Main responsibilities:

- Receive user commands from the UI.
- Forward page requests to the SimulationEngine.
- Control the simulation loop (next step, run, reset).
- Request UI updates after each step.

4.2.2 SimulationEngine

Role: Core simulation logic (Model).

Main responsibilities:

- Maintain page table and frames.
- Detect hits and page faults.
- Invoke the selected ReplacementPolicy when memory is full.
- Update the StatisticsTracker.

4.2.3 PageTable and Frame

PageTable:

- Maps virtual pages to physical frames.
- Stores basic PTE info (present bit, frame number, dirty/reference bits).

Frame:

- Represents a physical memory frame.
- Holds the current page ID and timing/usage info for FIFO/LRU.

4.2.4 ReplacementPolicy

Role: Strategy interface for page replacement.

Subclasses:

- **FIFOAlgorithm** – evicts the oldest loaded page.
- **LRUAlgorithm** – evicts the least recently used page.
- **OptimalAlgorithm** – evicts the page used farthest in the future.

The SimulationEngine calls `selectVictim()` when a new page must be loaded into full memory.

4.2.5 StatisticsTracker

Role: Collect performance metrics.

Main responsibilities:

- Count page hits and faults.
- Compute hit/miss ratio.
- Provide data for charts or summaries in the UI.

4.2.6 UserInterface

Role: Visual front-end (View).

Main responsibilities:

- Display frames, page table, and statistics.
- Highlight hits, faults, and replacements.
- Let the user enter reference strings, choose algorithms, and control the simulation.

4.3 Component Interconnection and Data Flow

4.3.1 Interaction Summary

The main flow is:

1. **UserInterface** sends commands to the **SimulationController**.
2. **SimulationController** forwards page requests to the **SimulationEngine**.
3. **SimulationEngine** uses **PageTable**, **Frame**, and **ReplacementPolicy**.
4. Engine updates state and **StatisticsTracker**.
5. **SimulationController** triggers a UI refresh.

4.3.2 Logical Structure

$$\text{UserInterface} \leftrightarrow \text{SimulationController} \leftrightarrow \text{SimulationEngine}$$

Inside the engine:

$$\text{SimulationEngine} \rightarrow (\text{PageTable}, \text{Frame}, \text{ReplacementPolicy}, \text{StatisticsTracker})$$

This design ensures:

- UI is independent from memory management logic.
- Replacement algorithms are interchangeable.
- The Model encapsulates internal state while staying easy to visualize.

5. Implementation

5.1 Code Organization

The simulator is implemented in Python and follows a modular design. The core logic resides in the `simulator` package, which interacts with the `tests` package for validation. The main modules are:

- `vm_config.py` – defines the `VMConfig` class, storing system parameters such as memory sizes and offset bits.
- `frame.py` – defines the `Frame` class representing a physical memory frame.
- `page_table.py` – implements the `PageTable` and `PageTableEntry` components.
- `tlb.py` – implements the `TLB` (Translation Lookaside Buffer) with its own LRU eviction logic.
- `simulation_engine.py` – the core engine orchestrating the MMU logic, including address translation, TLB interaction, and replacement strategies.
- `statistics_tracker.py` – a dedicated component for collecting metrics like hit/miss ratios, TLB performance, and disk writes.
- `replacement_policies/` – a subpackage containing the abstract `ReplacementPolicy` strategy and concrete implementations (FIFO, LRU, Optimal).

5.2 Configuration and Frame Representation

The system is initialized using a `VMConfig` object, which ensures consistency across components. It calculates derived properties:

$$\text{Page Size} = 2^{\text{offset\_bits}}$$

$$\text{Total Pages} = \frac{\text{Virtual Memory Size}}{\text{Page Size}}, \quad \text{Total Frames} = \frac{\text{Physical Memory Size}}{\text{Page Size}}$$

Physical memory is modelled as a list of `Frame` objects. Each frame tracks:

- **index**: its physical address.
- **page**: the virtual page number it currently holds.
- **loaded_time**: timestamp for FIFO replacement.
- **last_access_time**: timestamp for LRU replacement.

5.3 Memory Management Unit (MMU) Logic

The `SimulationEngine` class acts as the MMU. It processes one memory reference at a time via the `step()` method. The complete traversal path is:

1. **Address Translation**: The engine splits the Virtual Address into a `VPN` (Virtual Page Number) and `Offset`.
2. **TLB Lookup**: It checks the `TLB`.
   - If found (TLB Hit), the frame is retrieved immediately.
   - If missing (TLB Miss), it consults the Page Table.
3. **Page Table Lookup**:
   - If the page is `present`, it is a Page Hit. The TLB is updated with the new mapping.
   - If not present, it triggers a **Page Fault**.
4. **Page Fault Handling**:
   - The engine looks for a free frame.
   - If memory is full, it invokes the selected `ReplacementPolicy` to choose a victim.
   - **Write Back**: If the victim page is dirty (modified), a disk write is recorded.
   - The new page is loaded, and the Page Table and TLB are updated.

5.4 Translation Lookaside Buffer (TLB)

The simulator includes a realistic TLB implementation. It is a separate cache with a limited size (e.g., 4 entries). It uses its own internal LRU mechanism to evict the oldest entry when full, distinct from the main memory replacement policy. This adds an extra layer of realism, allowing students to observe the difference between TLB thrashing and main memory thrashing.

5.5 Replacement Policies

The system implements three policies using a Strategy pattern:

FIFO Replaces the page with the earliest `loaded_time`.

LRU Replaces the page with the oldest `last_access_time`. Correctly handles the "reference string vs page number" distinction to ensure accurate tracking.

Optimal Looks ahead in the reference string to find the page used farthest in the future.

5.6 Statistics and Results

A `StatisticsTracker` runs alongside the engine. It provides real-time metrics:

- **TLB Hit Ratio**: $\frac{\text{TLB Hits}}{\text{Total Accesses}}$
- **Page Fault Ratio**: $\frac{\text{Page Faults}}{\text{Total Accesses}}$
- **Disk Writes**: Total number of times a modified page was evicted (write-back).

5.7 User Interface Implementation

The user interface is built using the `pygame` library, providing a responsive and interactive graphical environment. The `MemoryVisualizer` class within `ui/gui.py` acts as the View component of the system. Its key responsibilities include:

- **Input Management**: Parses and validates user inputs for memory settings and reference strings, supporting both "Read" and "Write" operations.
- **Visualization Components**:
  - **Physical Memory View**: dynamically draws frames, highlighting active and victim pages with color-coding (e.g., green for access, red for eviction).
  - **Page Table View**: lists all virtual pages with their current status (Present, Dirty, Referenced).
  - **TLB View**: displays cached translations and indicates hits/misses.
- **Statistics Panel**: aggregates and displays real-time metrics from the `StatisticsTracker`, updating dynamically as the simulation progresses.

The logical flow connects user actions (button clicks) to the `SimulationController`, which advances the engine state and triggers a redraw of the UI, ensuring the visual representation is always synchronized with the underlying model.

5.8 Key Implementation Details and Challenges

Developing a simulator that is both mathematically accurate and visually intuitive presented several technical challenges. This section details the specific solutions adopted to address them.

5.8.1 Decoupling Logic from Visualization

A primary design goal was to ensure the simulation engine could run independently of the user interface. This was achieved by introducing the `SimulationStepResult` data class. Instead of the engine writing directly to the screen, it returns a comprehensive "snapshot" object after every step. This object contains:

- The current state of the page table and frames.
- Flags indicating if a hit, fault, or TLB miss occurred.
- Specific indices of the accessed frame and, if applicable, the victim frame.
- Boolean flags for write-back events.

This architectural decision allowed for the creation of a robust unit test suite (Chapter 6) that validates the internal state without needing to instantiate a graphical window.

5.8.2 Simulating Time for LRU

The Least Recently Used (LRU) algorithm requires precise knowledge of when each page was last accessed. In a real operating system, this is often approximated using hardware reference bits or a logical clock. In this simulator, we implemented a precise **logical clock** mechanism. The `SimulationEngine` maintains a `current_step` counter. Every time a page is accessed (whether via TLB or Page Table), the corresponding `Frame` object updates its `last_access_time` to the current step value. This guarantees that the LRU policy can always deterministically identify the true least recently used page, resolving potential ties via frame index.

5.8.3 Maintaining TLB Coherence

A critical realism aspect found in this project is the management of the Translation Lookaside Buffer (TLB). In many simple simulators, the TLB is treated as an infinite or independent cache. However, in our implementation, we enforce **coherence** between the TLB and Main Memory. When a page is evicted from Physical Memory (due to a page fault), any corresponding entry in the TLB must also be invalidated immediately. Failing to do so would allow the CPU to access a page that is no longer resident in RAM, leading to data corruption simulation errors. Our `SimulationEngine` explicitly handles this by checking the victim page against the TLB contents during every eviction cycle, a detail often overlooked in basic educational tools.

6. Testing and Validation

Reliability and correctness are paramount for a simulation tool intended for educational purposes. To ensure the simulator accurately models virtual memory behavior, a comprehensive test suite was developed using Python's standard `unittest` framework. The testing strategy focuses on unit testing individual components in isolation followed by integration testing of the complete simulation engine.

6.1 Test Suite Structure

The test suite is organized into separate modules, each testing a specific component of the simulator. Below is a detailed explanation of each test file, including the inputs and expected outputs.

6.1.1 Configuration Testing (`test_vm_config.py`)

This module verifies that the system correctly calculates derived parameters (page size, number of frames, total virtual pages) from the basic configuration values.

Test Case 1: Small Memory Config

- **Input**:
  - Virtual Memory: 256 bytes
  - Physical Memory: 64 bytes
  - Offset Bits: 4 (Page Size = $2^4 = 16$ bytes)
- **Expected Output**:
  - Page Size: 16
  - Number of Frames: $64/16 = 4$
  - Number of Virtual Pages: $256/16 = 16$

Test Case 2: Standard Memory Config

- **Input**:
  - Virtual Memory: 65536 bytes
  - Physical Memory: 4096 bytes
  - Offset Bits: 8 (Page Size = 256 bytes)
- **Expected Output**:
  - Page Size: 256
  - Number of Frames: 16
  - Number of Virtual Pages: 256

6.1.2 Page Table Logic (`test_page_table.py`)

This module tests the creation, retrieval, and status management of Page Table Entries (PTEs).

Test Case 1: Get or Create New Entry

- **Input**: Request for Page 10
- **Expected Output**: A new PTE with `present=False`, `dirty=False`, `referenced=False`, and `frame_index=None`.

Test Case 2: Get Existing Entry

- **Input**: Create Page 5, set `present=True`, then retrieve Page 5 again.
- **Expected Output**: The returned PTE is the same object instance and has `present=True`.

6.1.3 TLB Behavior (`test_tlb.py`)

This module validates the Translation Lookaside Buffer, specifically its lookup mechanism and LRU eviction policy.

Test Case 1: Insert and Lookup

- **Input**: Insert (Page 1 $\rightarrow$ Frame 10), then Lookup Page 1.
- **Expected Output**: Returns Frame 10. Lookup for a non-existent Page 2 returns `None`.

Test Case 2: LRU Eviction

- **Input**: TLB Size = 3.
  1. Insert Page 1, Page 2, Page 3.
  2. Insert Page 4 (triggers eviction).
- **Expected Output**: Page 1 (the oldest) is removed. Lookup(Page 1) returns `None`. Lookup(Page 2, 3, 4) succeeds.

Test Case 3: Lookup Updates Access Time

- **Input**: TLB Size = 3.
  1. Insert Page 1, Page 2, Page 3.
  2. Lookup Page 1 (refreshes its timestamp).
  3. Insert Page 4.
- **Expected Output**: Page 2 is evicted (since Page 1 was just used). Lookup(Page 1) still succeeds.

6.1.4 Replacement Algorithms (`test_policies.py`)

This module tests the core page replacement logic against a standard reference string to ensure algorithmic correctness.

Common Input for All Policies:

- **Reference String**: `[7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2]`
- **Number of Frames**: 3

Test Case 1: FIFO (First-In, First-Out)

- **Description**: Evicts the page submitted earliest.
- **Expected Output**: Reads: 13, Hits: 3, Faults: 10.

Test Case 2: LRU (Least Recently Used)

- **Description**: Evicts the page not used for the longest time.
- **Expected Output**: Reads: 13, Hits: 4, Faults: 9.

Test Case 3: Optimal

- **Description**: Evicts the page used furthest in the future.
- **Expected Output**: Reads: 13, Hits: 6, Faults: 7.

6.1.5 Simulation Engine Integration (`test_engine.py`)

This module tests the integration of the MMU, Page Table, and Replacement Policy.

Test Case 1: Step Execution and Address Translation

- **Input**: Config with Page Size 16. Reference String: `(0, "R"), (16, "R"), (0, "W")`
- **Expected Output**:
  1. Access 0 (Page 0) → Fault, loaded into Frame 0.
  2. Access 16 (Page 1) → Fault, loaded into Frame 1.
  3. Access 0 (Page 0) → Hit, and Page 0 is marked `dirty`.

Test Case 2: Eviction and Write-Back

- **Input**:
  - Small memory (only frames available).
  - Fill memory, dirty one page, then force eviction.
- **Expected Output**: The step result indicates `fault=True` and `write_back=True`, confirming the system correctly identified that the evicted page needed to be saved to disk.

6.2 Execution and Results

The entire test suite can be executed from the project root directory using the following command:

```
python -m unittest discover tests
```

Results: All 16 implementations tests currently pass successfully. This confirms that:

1. The MMU correctly handles hits, faults, and TLB updates.
2. Replacement policies accurately follow their respective algorithms.
3. Statistics (hits, faults, disk writes) are tracked without error.

7. Conclusions

7.1 Summary of Contributions

The primary goal of this project was to demystify the complex internal operations of virtual memory management systems. By designing and implementing a full-featured graphical simulator, we have created a tool that successfully bridges the gap between abstract textbook theory and concrete, observable behavior.

The resulting application offers a complete simulation pipeline:

1. **Input Flexibility**: Users can define arbitrary physical and virtual memory sizes, allowing them to explore edge cases like small memory constraints or large address spaces.

2. **Transparent Component Interaction**: The simulator visualizes the handshake between the CPU, TLB, MMU, and Physical Memory in real-time. Unlike static diagrams, users watch the data flow step-by-step.

3. **Algorithmic Depth**: The inclusion of FIFO, LRU, and Optimal algorithms provides a rigorous platform for comparative analysis. Users can run identical reference strings against different algorithms to empirically verify theoretical performance characteristics.

7.2 Educational Value and Impact

Computer architecture concepts are notoriously difficult to master because they occur at a microscopic scale and at nanosecond speeds, invisible to the human eye. This simulator serves as a "virtual microscope" for the operating system.

Visualizing the Invisible: The most significant contribution of this project is the visualization of the "dirty bit" lifecycle. In a standard lecture, students are told that modified pages require a disk write upon eviction. In our simulator, this is explicitly visually represented: pages turn a distinct color when written to, and a "Disk Write" counter increments only when a dirty page is chosen as a victim. This creates a lasting mental model of the cost of memory writes versus reads.

Understanding Thrashing: By allowing users to configure small physical memory sizes (e.g., 4 frames), the simulator makes it easy to demonstrate "thrashing"—a state where the system spends more time paging than executing. Students can observe the high frequency of red "Page Fault" indicators and the rapid turnover of frame contents, providing a visceral understanding of why memory capacity is critical for system performance.

Belady's Anomaly: The flexibility to change the number of frames and the reference string allows advanced students to reproduce Belady's Anomaly using the FIFO algorithm. This transforms a theoretical curiosity into a reproducible scientific experiment.

7.3 Technical Challenges and Lessons Learned

The development process highlighted several key engineering challenges inherent in simulation software.

State Synchronization: One of the most persistent difficulties was ensuring that the visual state (the UI) never lagged behind or contradicted the logical state (the Engine). Early iterations struggled with "ghost" entries in the TLB that persisted after a page was evicted from RAM. This was resolved by implementing strict state validation checks and the "Snapshot" design pattern described in Chapter 5.8, ensuring the UI always renders a truthful representation of the backend data.

Balancing Realism and Simplicity: A design trade-off had to be made regarding the "present bit" and "valid bit". While real page tables contain dozens of control bits (protection, user/supervisor, etc.), we chose to focus strictly on *Present*, *Dirty*, and *Reference* bits. This simplification prevents overwhelming the user while preserving the core logic necessary to understand page replacement.

Event-Driven Simulation: Moving from a command-line script to a Pygame-based GUI required a shift in thinking from procedural execution to an event loop architecture. Handling user interrupts (pause, reset) while a simulation "auto-run" loop is active required careful management of thread-like behavior within the single-threaded Pygame main loop.

7.4 Limitations and Future Directions

While the current system is robust and feature-complete for its initial scope, there are several avenues for future expansion to increase its depth and utility.

Advanced Replacement Algorithms: Currently, the system supports the "Big Three" (FIFO, LRU, Optimal). Future versions could implement:

- **Second Chance / Clock Algorithm**: This would require visualizing the "circular buffer" nature of the clock hand, a highly instructive visual aid.
- **LFU (Least Frequently Used)**: Demonstrating the "count" overhead.

Multi-Level Paging: The current implementation uses a flat (linear) page table. For large virtual memory configurations, this is inefficient. Implementing a two-level or multi-level page table visualization would allow the tool to teach modern x86-64 memory architecture concepts, showing how sparse address spaces are managed efficiently.

Per-Process Simulation: At present, the simulator models a single reference stream effectively representing one process. Adding support for multiple reference streams (multitasking) would introduce the complexity of "Global" vs. "Local" page replacement scopes, a critical topic in OS design.

Variable Page Sizes: Supporting "Huge Pages" (e.g., mixing 4KB and 2MB pages) would demonstrate how TLB coverage can be improved, a relevant topic in high-performance computing optimization.

In summary, the Virtual Memory Simulator project has evolved from a simple concept into a powerful, interactive educational platform. It not only fulfills the technical requirements of accurately modeling memory management but also succeeds in its broader mission: making the invisible visible, and the complex understandable.

8. References

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2020). *Operating System Concepts*. Wiley.

[2] Stallings, W. (2018). *Operating Systems: Internals and Design Principles*. Pearson.

[3] Tanenbaum, A. S. (2014). *Modern Operating Systems*. Pearson.